SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

PROJECT 38: DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR EMPLOYEE ATTENDANCE, SHIFT AND LEAVE MANAGEMENT SYSTEM

Program	B.Tech (AI and ML)
Section	Leopards
Faculty	Dr. C. Nagaraju
Date of Submission	12-10-2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Email ID
25WU0102109	Joshitha Chintakrinda	Joshitha.chintakrinda_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Dr. C. Nagaraju, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **"Design and Implementation of a Database Management System for Employee Attendance, Shift and Leave Management System"**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date	:			**Signature**
Place	:		**Name:**	Joshitha Chintakrinda
			Roll Number:	25WU0102109
			Program:	B.Tech (AI and ML)

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	12

1.7 Entity and Relationship Summary	13
2. LOGICAL DATABASE DESIGN	14
2.1 Relational Schema	14
2.2 Keys and Integrity Constraints	14
2.3 Functional Dependencies	15
2.4 Normalization up to Third Normal Form (3NF)	16
2.5 Data Dictionary	17
2.6 Implementation of Logical Design: DDL and Sample Data	19
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	21
3.1 Application Overview	21
3.2 Development Environment and Database Connectivity	21
3.3 Application Modules and Database Mapping	21
3.4 CRUD Operations	22
3.5 Business Rules, Validation and Error Handling	22
3.6 Reports and Database Operations	23
3.7 Application Screens	23
3.8 Testing and Results	25
4. CONCLUSION	27
4.1 Limitations and Future Enhancements	27
5. REFERENCES	28
6. APPENDICES	29

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	11
Table 4	Entity and Relationship Summary	13
Table 5	Relational Schema	14
Table 6	Keys and Integrity Constraints	14
Table 7	Normalization Status up to 3NF	16

Table 8	Data Dictionary	17
Table 9	Database Implementation Environment	19
Table 10	Application Development Environment	21
Table 11	Application Module-to-Database Mapping	21
Table 12	CRUD Operations	22
Table 13	Business Rules, Validation and Error Handling	22
Table 14	Reports and Database Operations	23
Table 15	Test Cases	25

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Roster Preparation and Overlapping-Shift Rejection	24
Figure 3	Clock-in, Clock-out and Work-Hour Calculation	24
Figure 4	Leave Processing and Balance Update	24
Figure 5	Attendance Reports: Lateness and Department Summary	25

LIST OF ACRONYMS

Acronym	Expansion
3NF	Third Normal Form
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FD	Functional Dependency
FK	Foreign Key
FR	Functional Requirement
HR	Human Resources
IEEE	Institute of Electrical and Electronics Engineers
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language

ABSTRACT

Shift-based organisations depend on accurate rosters, attendance punches, overtime and leave records, but consolidating these manually leads to staffing gaps, payroll errors and disputes. This project designs and implements a relational database for an Employee Attendance, Shift and Leave Management System. The objective is to store, validate and report workforce data reliably while enforcing the business rules of the organisation inside the database. An EntityRelationship model of thirteen entities covering employees, departments, shifts, rosters, attendance punches, daily summaries, overtime requests, leave types, balances, applications, approvals and holidays was converted into a relational schema and normalised to Third Normal Form. The schema was implemented in SQLite with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints, and triggers that prevent overlapping shifts, clock-out before clock-in, leave beyond balance and unauthorised overtime. A Python application connects to the database to prepare rosters, record clock-in and clock-out, calculate work hours, process overtime and leave approvals within transactions, and generate reports on rosters, lateness, absence, overtime, leave balances, shift coverage and department summaries. Fourteen test cases, including negative cases, were executed and all produced the expected outcome. The system shows how a well-designed database improves data integrity and supports payroll-ready attendance reporting.

Keywords

Keywords: Relational Database, ER Diagram, Normalization, Attendance Management, Shift Roster, Leave Management, SQL, Python

CHAPTER 1 PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

This chapter presents the problem addressed by Project 38, the objectives, the scope, the proposed solution and the approved ER diagram.

1.1 Problem Statement

Organisations that run shifts, such as manufacturing plants, call centres and support desks, must know who is scheduled, who actually reported, how long each person worked, and how much leave each person is still entitled to. When these records are kept in separate registers or spreadsheets, rosters are consolidated by hand. Employees can be double-booked, clock-out times can be recorded before clock-in times, overtime can be paid without approval, and leave can be granted beyond the available balance. These errors create staffing gaps and payroll disputes, and they are difficult to trace because no single record is authoritative.

The affected users are employees, who need to punch attendance and apply for leave; managers, who prepare rosters and approve overtime and leave; and HR and payroll staff, who need reliable summaries. A relational database [1] is needed so that every fact is stored once, relationships are enforced, and business rules are checked by the database itself rather than by individual users. This project therefore designs and implements such a database together with a small application that uses it.

1.2 Objectives

- Design an ER model and a relational schema for employees, departments, shifts, rosters, attendance, overtime, leave, approvals and holidays.
- Normalise the schema to Third Normal Form and document keys, functional dependencies and a data dictionary.
- Implement the schema with integrity constraints and triggers that enforce no overlapping shifts, clock-out after clock-in, leave within balance, authorised overtime and one daily summary per employee.
- Develop a Python application that performs roster preparation, clock-in/clock-out, work-hour calculation, overtime approval, leave processing, balance update and reporting using parameterised SQL.
- Validate the system through positive and negative test cases and report the results.

1.3 Scope and Assumptions

Table 1 lists what the system covers, what it excludes and the assumptions made during design.

Table 1. Scope and Assumptions

Item	Description
In Scope	Employee and department master data; shift definitions and daily rosters; clockin/clock-out punches; daily attendance summary (worked time, lateness, overtime, status); overtime request and approval; leave types, yearly balances, applications and approvals; holiday calendar; role-based user records; reports for rosters, lateness, absence, overtime, leave balance, shift coverage and department summary.
Out of Scope / Exclusions	Payroll calculation and salary slips; biometric device integration (punch source is recorded but no device driver is provided); mobile or web front end; multicompany or multi-currency support; password hashing and authentication workflow beyond storing user roles.
Assumptions	Each employee works at most one rostered shift per calendar day; night shifts end after midnight on the following day; one year of leave is tracked per balance row; a manager approves requests for his or her team; times are recorded in a single local time zone; the grace period for lateness is defined per shift.

1.4 Proposed Solution Strategy

The solution is a normalised relational database that enforces the organisation's attendance and leave rules, together with a Python application that performs daily operations through parameterised SQL. Table 2 summarises the users, modules, workflow, rules and outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Employee (punch, apply leave, request overtime), Manager (prepare roster, approve overtime and leave), Admin / HR (maintain master data, run reports).
Major Application Modules	Employee maintenance; roster preparation; clock-in/clock-out; work-hour calculation and daily summary; overtime request and approval; leave application, approval and balance update; reports.
Proposed Workflow	Admin creates departments, employees and shifts → manager assigns daily roster → employee clocks in and out → system computes worked time, lateness and approved overtime into the daily summary → employee applies for leave or overtime → manager approves or rejects → approved leave reduces the balance in the same transaction → HR runs reports.
Important Business Rules	(1) No overlapping shifts for an employee. (2) Clock-out must be after clock-in. (3) Leave days must not exceed the remaining balance. (4) Overtime is recorded only when an approved request exists. (5) Only one daily summary per employee per date.
Expected Reports / Outputs	Weekly roster, lateness report, absence report, overtime summary, leave balance, shift coverage, department summary and a list of above-average workers.

1.5 Functional Requirements

Table 3 lists the functional requirements that the database and application must satisfy.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Maintain employee and department records, including manager and active/inactive status.	High
FR-02	Define shifts with start time, end time and grace period.	High
FR-03	Prepare rosters by assigning an employee to a shift for a date, rejecting overlapping assignments.	High
FR-04	Record clock-in and clock-out punches and reject clock-out earlier than clock-in.	High
FR-05	Calculate worked minutes, lateness and status for each day and store one daily summary.	High
FR-06	Accept overtime requests and record overtime only after approval.	High

FR-07	Accept leave applications by leave type and reject those exceeding the balance.	High
FR-08	Record approval decisions and update leave balance atomically.	High
FR-09	Maintain the holiday calendar and user roles.	Medium
FR-10	Generate roster, lateness, absence, overtime, leave balance, coverage and department reports.	High

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the ER diagram. Following the entity-relationship approach [2], it presents each entity with its attributes, primary keys (PK), foreign keys (FK) and unique attributes (U), and the cardinality of every relationship. The holiday entity is independent and is used by roster and absence checks.

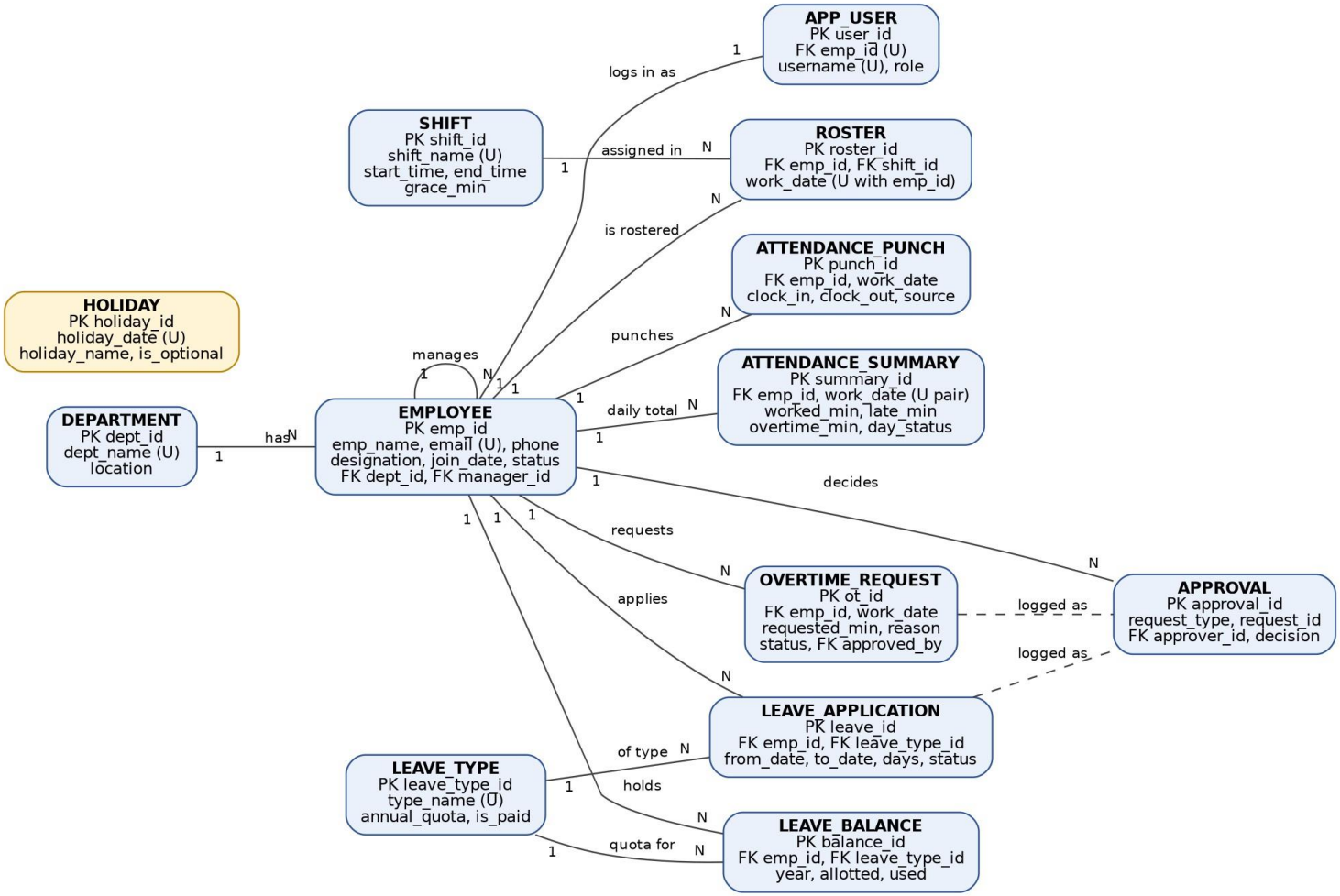


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 summarises each entity with its major attributes and cardinality.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
DEPARTMENT	dept_id, dept_name, location. 1 department : N employees	Groups employees; name must be unique
EMPLOYEE	emp_id, emp_name, email, designation, join_date, status, dept_id, manager_id. Self-relationship manages (1:N)	Master record of every worker; manager is another employee
APP_USER	user_id, emp_id, username, role. 1 employee : 1 user	Login identity with ADMIN, MANAGER or EMPLOYEE role
SHIFT	shift_id, shift_name, start_time, end_time, grace_min	Defines working hours; night shift ends after midnight
ROSTER	roster_id, emp_id, shift_id, work_date. Employee 1:N roster, shift 1:N roster	Planned shift of an employee for a date; no overlap allowed
ATTENDANCE_PUNCH	punch_id, emp_id, work_date, clock_in, clock_out, source. Employee 1:N punches	Raw clock-in and clock-out events
ATTENDANCE_SUMMA RY	summary_id, emp_id, work_date, worked_min, late_min, overtime_min, day_status. Employee 1:N	One summary per employee per date
OVERTIME_REQUEST	ot_id, emp_id, work_date, requested_min, reason, status, approved_by	Overtime must be requested and approved before it counts
LEAVE_TYPE	leave_type_id, type_name, annual_quota, is_paid	Casual, Sick, Earned and Loss of Pay leave
LEAVE_BALANCE	balance_id, emp_id, leave_type_id, year, allotted, used. Employee 1:N, leave type 1:N	Yearly entitlement and usage; used cannot exceed allotted
LEAVE_APPLICATION	leave_id, emp_id, leave_type_id, from_date, to_date, days, status	Leave request; days must fit the balance
APPROVAL	approval_id, request_type, request_id, approver_id, decision, remarks, decided_on. Approver (employee) 1:N	Audit trail of every leave or overtime decision
HOLIDAY	holiday_id, holiday_date, holiday_name, is_optional	Public and optional holidays used for roster and absence checks

The relationships are: department–employee (1:N), employee–employee manager (1:N), employee–app_user (1:1), employee–roster (1:N), shift–roster (1:N), employee–attendance_punch (1:N), employee–attendance_summary (1:N), employee–overtime_request (1:N), employee–leave_balance (1:N), leave_type–leave_balance (1:N), employee– leave_application (1:N), leave_type–leave_application (1:N) and employee–

approval (1:N). Every approval row refers to either a leave application or an overtime request through request_type and request_id.

CHAPTER 2 LOGICAL DATABASE DESIGN

This chapter documents how the approved ER model was converted into a relational design, normalised and implemented as a working database prototype.

2.1 Relational Schema

Each entity of Figure 1 became one relation; the one-to-many relationships were implemented as foreign keys on the "many" side. Table 5 lists the thirteen relations derived from the ER model.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	department	Stores departments and their locations.
2	employee	Stores employee master data with department and manager.
3	app_user	Stores application users and roles linked to employees.
4	shift	Stores shift timings and grace period.
5	roster	Stores the shift assigned to an employee on a date.
6	attendance_punch	Stores clock-in and clock-out events.
7	attendance_summary	Stores the computed daily summary for each employee.
8	overtime_request	Stores overtime requests and their status.
9	leave_type	Stores leave categories and annual quotas.
10	leave_balance	Stores yearly allotted and used leave per type.
11	leave_application	Stores leave requests of employees.
12	approval	Stores approval decisions for leave and overtime.
13	holiday	Stores the holiday calendar.

2.2 Keys and Integrity Constraints

Table 6 shows the primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints and the triggers used to enforce the business rules. Foreign keys use ON DELETE RESTRICT where history must be protected, ON DELETE CASCADE for dependent child rows, and ON DELETE SET NULL for optional references.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
department	Primary Key	dept_id	Unique identity of a department

	Unique / Default	dept_name; location	No duplicate department names; location defaults to Main Campus
employee	Primary Key	emp_id	Unique identity of an employee
Table Name	**Key / Constraint**	**Attribute(s)**	**Business Rule Enforced**
	Foreign Key	dept_id → department; manager_id → employee	Employee belongs to a valid department; ON DELETE RESTRICT protects department data
	Unique / Check	email; status	One e-mail per employee; status limited to ACTIVE or INACTIVE
app_user	Foreign Key / Unique	emp_id (unique); username (unique)	One login per employee
	Check	role	ADMIN, MANAGER or EMPLOYEE only
shift	Unique / Check	shift_name; grace_min >= 0	Unique shift names; non-negative grace period
roster	Foreign Key	emp_id, shift_id	Roster refers to existing employee and shift
	Unique Index + Trigger	(emp_id, work_date); trg_roster_no_overlap	No overlapping shifts: one shift per day and no clash with previous night shift
attendance_punch	Check + Trigger	clock_out > clock_in; trg_punch_out_after_in	Clock-out after clock-in
	Foreign Key / Default	emp_id; source	Valid employee; source defaults to APP
attendance_summary	Unique	(emp_id, work_date)	One daily summary per employee
	Check + Trigger	day_status; trg_overtime_authorised	Valid status; overtime only with an approved request
overtime_request	Check / Default	requested_min between 1 and 240; status	Limits overtime request size; status defaults to PENDING
leave_type	Unique / Check	type_name; annual_quota >= 0	Unique types, valid quota
leave_balance	Unique / Check	(emp_id, leave_type_id, year); used <= allotted	One balance row per type and year; usage cannot exceed allotment
leave_application	Check + Trigger	to_date >= from_date; days > 0; trg_leave_within_balance	Valid dates; leave within balance

approval	Foreign Key / Check	approver_id; request_type; decision	Approver is an employee; decision APPROVED or REJECTED
holiday	Unique	holiday_date	One holiday record per date

2.3 Functional Dependencies

The important functional dependencies [3] used to justify normalization are listed below.

```
dept_id -> dept_name, location
emp_id -> emp_name, email, phone, designation, dept_id, manager_id,
    join_date, status email -> emp_id   (candidate key)
shift_id -> shift_name, start_time, end_time, grace_min
```

```
roster_id -> emp_id, shift_id, work_date
(emp_id, work_date) -> roster_id, shift_id   (one shift per day)
punch_id -> emp_id, work_date, clock_in, clock_out, source
(emp_id, work_date) -> worked_min, late_min, overtime_min, day_status
    [attendance_summary]
ot_id -> emp_id, work_date, requested_min, reason, status, approved_by
leave_type_id -> type_name, annual_quota, is_paid
(emp_id, leave_type_id, year) -> allotted, used
leave_id -> emp_id, leave_type_id, from_date, to_date, days, reason, status
approval_id -> request_type, request_id, approver_id, decision, remarks,
    decided_on
holiday_date -> holiday_name, is_optional
```

2.4 Normalization up to Third Normal Form (3NF)

Normalization [4], [5] removes redundancy and update anomalies. The initial design kept employee, department, shift and leave details in one attendance register. This caused repetition of department names and shift timings and allowed inconsistent values. The register was decomposed so that every non-key attribute depends on the key, the whole key and nothing but the key. All relations satisfy 1NF because every attribute is atomic. Relations with composite candidate keys, such as attendance_summary (emp_id, work_date) and leave_balance (emp_id, leave_type_id, year), have no partial dependencies, so they satisfy 2NF. No relation has a transitive dependency, so all satisfy 3NF; because every determinant listed in Section 2.3 is a candidate key, they also satisfy BCNF. Table 7 gives the status of every relation.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
department	Yes	Yes	Yes	Yes	Department details separated from employee to remove the transitive dependency emp_id → dept_id → dept_name.
employee	Yes	Yes	Yes	Yes	Single-column key; no non-key attribute depends on another non-key attribute.
app_user	Yes	Yes	Yes	Yes	Login data separated from employee data.
shift	Yes	Yes	Yes	Yes	Shift timings stored once and referenced by roster.

roster	Yes	Yes	Yes	Yes	Stores only keys and date; shift timings are not repeated.
attendance_punch	Yes	Yes	Yes	Yes	Atomic punch events; no derived data.
attendance_summary	Yes	Yes	Yes	Yes	Deliberately stored derived totals per date for reporting; every attribute depends on the whole key (emp_id, work_date).
overtime_request	Yes	Yes	Yes	Yes	Approver stored as a key, not as a name.
leave_type	Yes	Yes	Yes	Yes	Quota stored once per type.
leave_balance	Yes	Yes	Yes	Yes	Remaining balance not stored; derived in view v_leave_balance.
leave_application	Yes	Yes	Yes	Yes	Leave type referenced by key.
approval	Yes	Yes	Yes	Yes	Single audit relation for leave and overtime decisions.
holiday	Yes	Yes	Yes	Yes	Date determines name; no redundancy.

2.5 Data Dictionary

Table 8 describes every attribute of every table. Dates and times are stored as ISO text (YYYYMM-DD and YYYY-MM-DD HH:MM), which SQLite compares correctly.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
department	dept_id	INTEGER	PK, auto	Department identifier
	dept_name	TEXT	NOT NULL, UNIQUE	Department name
	location	TEXT	NOT NULL, DEFAULT	Work location
employee	emp_id	INTEGER	PK, auto	Employee identifier
	emp_name	TEXT	NOT NULL	Full name
	email	TEXT	NOT NULL, UNIQUE, CHECK	Official e-mail
	phone	TEXT		Contact number
	designation	TEXT	NOT NULL	Job title
	dept_id	INTEGER	FK, NOT NULL	Department of the employee
	manager_id	INTEGER	FK (self)	Reporting manager
	join_date	TEXT	NOT NULL	Date of joining
	status	TEXT	CHECK, DEFAULT	ACTIVE or INACTIVE

app_user	user_id	INTEGER	PK, auto	User identifier
	emp_id	INTEGER	FK, UNIQUE	Linked employee
	username	TEXT	UNIQUE, NOT NULL	Login name
	password_hash	TEXT	NOT NULL	Stored password hash
	role	TEXT	CHECK	ADMIN, MANAGER, EMPLOYEE
shift	shift_id	INTEGER	PK, auto	Shift identifier
	shift_name	TEXT	UNIQUE	Name such as Morning
	start_time	TEXT	NOT NULL	Shift start (HH:MM)
	end_time	TEXT	NOT NULL	Shift end (HH:MM)
	grace_min	INTEGER	CHECK >= 0	Minutes allowed before late
roster	roster_id	INTEGER	PK, auto	Roster identifier
	emp_id	INTEGER	FK	Rostered employee
	shift_id	INTEGER	FK	Assigned shift
	work_date	TEXT	NOT NULL	Date of the shift
attendance_punch	punch_id	INTEGER	PK, auto	Punch identifier
	emp_id	INTEGER	FK	Employee
	work_date	TEXT	NOT NULL	Attendance date
	clock_in	TEXT	NOT NULL	Clock-in date and time
	clock_out	TEXT	CHECK > clock_in	Clock-out date and time

Table	**Attribute**	**Data Type**	**Key / Constraint**	**Description**
	source	TEXT	CHECK, DEFAULT	APP, BIOMETRIC or MANUAL
attendance_summary	summary_id	INTEGER	PK, auto	Summary identifier
	emp_id	INTEGER	FK, UNIQUE (pair)	Employee

	work_date	TEXT	UNIQUE (pair)	Date summarised
	worked_min	INTEGER	CHECK >= 0	Minutes worked
	late_min	INTEGER	CHECK >= 0	Minutes late
	overtime_min	INTEGER	CHECK >= 0	Approved overtime minutes
	day_status	TEXT	CHECK	PRESENT, LATE, ABSENT, ON_LEAVE, HOLIDAY, WEEKOFF
overtime_request	ot_id	INTEGER	PK, auto	Request identifier
	emp_id	INTEGER	FK	Requesting employee
	work_date	TEXT	NOT NULL	Date of overtime
	requested_min	INTEGER	CHECK 1-240	Minutes requested
	reason	TEXT	NOT NULL	Reason
	status	TEXT	CHECK, DEFAULT	PENDING, APPROVED, REJECTED
	approved_by	INTEGER	FK	Approving manager
leave_type	leave_type_id	INTEGER	PK, auto	Leave type identifier
	type_name	TEXT	UNIQUE	Casual, Sick, Earned, LOP
	annual_quota	INTEGER	CHECK >= 0	Days per year
	is_paid	INTEGER	CHECK 0/1	Paid leave flag
leave_balance	balance_id	INTEGER	PK, auto	Balance identifier
	emp_id	INTEGER	FK, UNIQUE (triple)	Employee
	leave_type_id	INTEGER	FK, UNIQUE (triple)	Leave type
	year	INTEGER	UNIQUE (triple)	Calendar year
	allotted	INTEGER	CHECK >= 0	Days allotted
	used	INTEGER	CHECK used <= allotted	Days used
leave_application	leave_id	INTEGER	PK, auto	Application identifier

	emp_id	INTEGER	FK	Applicant
	leave_type_id	INTEGER	FK	Leave type
	from_date	TEXT	NOT NULL	First day of leave
	to_date	TEXT	CHECK >= from_date	Last day of leave
	days	INTEGER	CHECK > 0	Number of days
	status	TEXT	CHECK, DEFAULT	PENDING, APPROVED, REJECTED
approval	approval_id	INTEGER	PK, auto	Approval identifier
	request_type	TEXT	CHECK	LEAVE or OVERTIME
Table	**Attribute**	**Data Type**	**Key / Constraint**	**Description**
	request_id	INTEGER	NOT NULL	Id of the request decided
	approver_id	INTEGER	FK	Deciding manager
	decision	TEXT	CHECK	APPROVED or REJECTED
	decided_on	TEXT	DEFAULT now	Decision timestamp
holiday	holiday_id	INTEGER	PK, auto	Holiday identifier
	holiday_date	TEXT	UNIQUE	Date
	holiday_name	TEXT	NOT NULL	Holiday name
	is_optional	INTEGER	CHECK 0/1	Optional holiday flag

2.6 Implementation of Logical Design: DDL and Sample Data

The schema was implemented in SQLite [6]. Table 9 gives the implementation environment. The complete script is in Appendix A; representative statements follow.

Table 9. Database Implementation Environment

Item	Details
DBMS	SQLite (file-based, foreign keys enabled with PRAGMA foreign_keys = ON)
DBMS Version	3.45.1
Number of Tables	13 tables, 2 views (v_daily_attendance, v_leave_balance), 4 triggers

Sample Data	4 departments, 8 employees, 4 shifts, 4 leave types, 24 leave balances, 4 holidays and a 25-row roster for 5 to 9 October 2026; punches, summaries, overtime and leave records generated by the application

Employee table with foreign keys and CHECK constraints:

```
CREATE TABLE employee (
    emp_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    emp_name    TEXT NOT NULL,
    email       TEXT NOT NULL UNIQUE CHECK (email LIKE '%_@_%._%'),
    phone       TEXT,
designation TEXT NOT NULL,
dept_id     INTEGER NOT NULL,
    manager_id           INTEGER,
join_date   TEXT NOT NULL,
    status      TEXT NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
        ('ACTIVE','INACTIVE')),
    FOREIGN KEY (dept_id) REFERENCES department(dept_id) ON UPDATE
        CASCADE ON DELETE RESTRICT,
    FOREIGN KEY (manager_id) REFERENCES employee(emp_id) ON DELETE
SET
        NULL
);
```

Attendance summary with the one-summary-per-day rule:

```
CREATE TABLE attendance_summary (
    summary_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    emp_id      INTEGER NOT NULL,
work_date   TEXT NOT NULL,
    worked_min  INTEGER NOT NULL DEFAULT 0 CHECK (worked_min >= 0),
    late_min    INTEGER NOT NULL DEFAULT 0 CHECK (late_min >= 0),
overtime_min INTEGER NOT NULL DEFAULT 0 CHECK (overtime_min >= 0),
    day_status  TEXT NOT NULL CHECK (day_status IN
        ('PRESENT','LATE','ABSENT','ON_LEAVE','HOLIDAY','WEEKOFF')),
    -- Rule: one daily summary per employee
    UNIQUE (emp_id, work_date),
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE
);
```

Triggers enforcing leave within balance and authorised overtime:

```
CREATE TRIGGER trg_leave_within_balance
BEFORE INSERT ON leave_application
BEGIN
    SELECT RAISE(ABORT, 'Insufficient leave balance')
    WHERE COALESCE((SELECT allotted - used FROM leave_balance
                    WHERE emp_id = NEW.emp_id
                      AND leave_type_id = NEW.leave_type_id
                      AND year = CAST(substr(NEW.from_date,1,4) AS
                          INTEGER)), 0) < NEW.days;
END;

CREATE TRIGGER trg_overtime_authorised
BEFORE INSERT ON attendance_summary
WHEN NEW.overtime_min > 0
BEGIN
    SELECT RAISE(ABORT, 'Overtime not authorised: no approved overtime
        request')
```

```
    WHERE NOT EXISTS (SELECT 1 FROM overtime_request o
                      WHERE o.emp_id = NEW.emp_id AND o.work_date =
                          NEW.work_date
                        AND o.status = 'APPROVED' AND o.requested_min >=
                            NEW.overtime_min);
END;
```

Representative sample data:

```
department : (1,'Operations','Plant A'), (2,'Human Resources','Head Office'),
(3,'IT Support','Head Office'), (4,'Customer Care','Call Centre')
employee   : (3,'Kiran Kumar','Machine Operator',dept 1), (4,'Sneha
    Reddy','Support Engineer',dept 3),
             (5,'Imran Khan','Support Agent',dept 4), ... 8 employees in total
shift      : (1,'Morning','06:00','14:00',10), (2,'General','09:00','17:00',10),
             (3,'Evening','14:00','22:00',10), (4,'Night','22:00','06:00',15)
leave_type : (1,'Casual Leave',12), (2,'Sick Leave',10), (3,'Earned Leave',15),
    (4,'Loss of Pay',30)
roster     : 25 rows - five employees on five days, 5 to 9 Oct 2026 holiday    :
2026-10-02 Gandhi Jayanti, 2026-10-20 Vijayadashami, 2026-11-08
    Diwali, 2026-12-25 Christmas
```

CHAPTER 3 APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

This chapter shows how the Python application uses the database. No data is held in the program; every operation reads or writes the database.

3.1 Application Overview

The application is a menu-driven Python program (app.py) used by employees, managers and HR. It lets users assign rosters, clock in and out, calculate the daily summary, request overtime, apply for leave, maintain employees and run reports. The work-hour calculation reads the punch and the rostered shift, computes worked minutes, lateness beyond the grace period and only the overtime that has an approved request, and writes one summary row. Leave approval changes the application status, writes the approval record and updates the balance inside a single transaction.

3.2 Development Environment and Database Connectivity

The program connects through the standard sqlite3 module [7]. A connection is opened with connect(), foreign keys are switched on, and rows are accessed by column name. All statements use ? placeholders so that user input is never concatenated into SQL. Table 10 lists the environment.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.12
DBMS	SQLite 3.45.1

IDE / Framework / GUI Toolkit	Any Python editor; console menu interface (no external framework)
Database Connector / Driver	sqlite3 (Python standard library)
Operating Environment	Ubuntu Linux; portable to Windows and macOS
Repository Link	Source files submitted with this report (see Appendix B)

3.3 Application Modules and Database Mapping

Table 11 maps every module to the tables it uses and the operations it performs.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Employee Maintenance	Add, search, update and deactivate employees	employee, department	INSERT, SELECT, UPDATE
Roster Preparation	Assign shifts to employees by date	roster, shift, employee	INSERT, DELETE, SELECT
Module / Screen	**Purpose**	**Tables Used**	**Main Operations**
Clock-in / Clock-out	Record punches	attendance_punch	INSERT, UPDATE
Work-Hour Calculation	Compute worked time, lateness, approved overtime	attendance_punch, roster, shift, attendance_summary, overtime_request	SELECT, INSERT
Overtime Approval	Request and decide overtime	overtime_request, approval	INSERT, UPDATE
Leave Processing	Apply, approve and update balance	leave_application, leave_balance, approval	INSERT, UPDATE (transaction)
Reports	Roster, lateness, absence, overtime, balance, coverage, summary	views and all core tables	SELECT with JOIN, GROUP BY, subquery

3.4 CRUD Operations

Table 12 shows how create, read, update and delete operations are covered, with the evidence in the screens and test cases.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Add employee; assign roster; clock-in; apply leave; request overtime	employee, roster, attendance_punch, leave_application, overtime_request	Figures 2 to 4; TC01, TC-12
Read / Search	Search employees; run reports	employee, department, all report tables	Figure 5; TC-12

Update	Change designation; clock-out; approve leave and update balance	employee, attendance_punch, leave_application, leave_balance	Figures 3 and 4; TC08, TC-13
Delete / Controlled Removal	Deactivate employee (history kept); remove a roster entry	employee, roster	TC-14

Employees are never physically deleted: deactivation keeps attendance and leave history intact, which is why the controlled-removal operation sets status to INACTIVE.

3.5 Business Rules, Validation and Error Handling

Rules are enforced in the database so that they hold regardless of which program writes the data. The application wraps each operation in a transaction, catches sqlite3.Error and ValueError, shows the message to the user and leaves the database unchanged. Table 13 lists the rules and the messages observed.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
No overlapping shifts	Unique index on (emp_id, work_date) and BEFORE INSERT trigger; the application catches the database error and shows it	ERROR: Overlapping shift: employee already rostered for this date
Rule / Validation	**How Implemented**	**Evidence / Message**
Clock-out after clockin	CHECK constraint and BEFORE UPDATE trigger	ERROR: Clock-out must be after clockin
Leave within balance	BEFORE INSERT trigger compares days with allotted - used	ERROR: Insufficient leave balance
Authorised overtime only	Trigger on attendance_summary; application records only approved minutes	ERROR: Overtime not authorised: no approved overtime request
One daily summary per employee	UNIQUE (emp_id, work_date)	ERROR: UNIQUE constraint failed: attendance_summary.emp_id, attendance_summary.work_date
Atomic approval	BEGIN ... COMMIT; ROLLBACK on error; status and balance change together	Second approval: Leave request not found or already processed
Safe SQL	All statements use ? placeholders (parameterised queries); errors caught as sqlite3.Error or ValueError	No SQL injection through input fields

3.6 Reports and Database Operations

Table 14 lists the reports and the SQL concepts used: joins, aggregate functions, GROUP BY with HAVING, views, correlated and nested subqueries.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output

Weekly Roster	JOIN of roster, employee, department, shift; ORDER BY	Appendix C
Lateness Report	View v_daily_attendance with WHERE late_min > 0	Figure 5
Absence Report	NOT EXISTS subquery against punches and approved leave	Appendix C
Overtime Summary	GROUP BY with SUM and COUNT	Appendix C
Leave Balance	View v_leave_balance (allotted - used)	Appendix C
Shift Coverage	GROUP BY with HAVING	Appendix C
Department Summary	Aggregate AVG, SUM with CASE and COUNT DISTINCT	Figure 5
Above-Average Workers	Nested query: worked_min > (SELECT AVG(...))	Appendix C

3.7 Application Screens

Figures 2 to 5 are captured from the running application using the sample data. Figure 2 shows roster entry and the rejection of an overlapping shift; Figure 3 shows clock-in, clock-out and the work-hour calculation; Figure 4 shows leave processing and the balance update; Figure 5 shows reports.

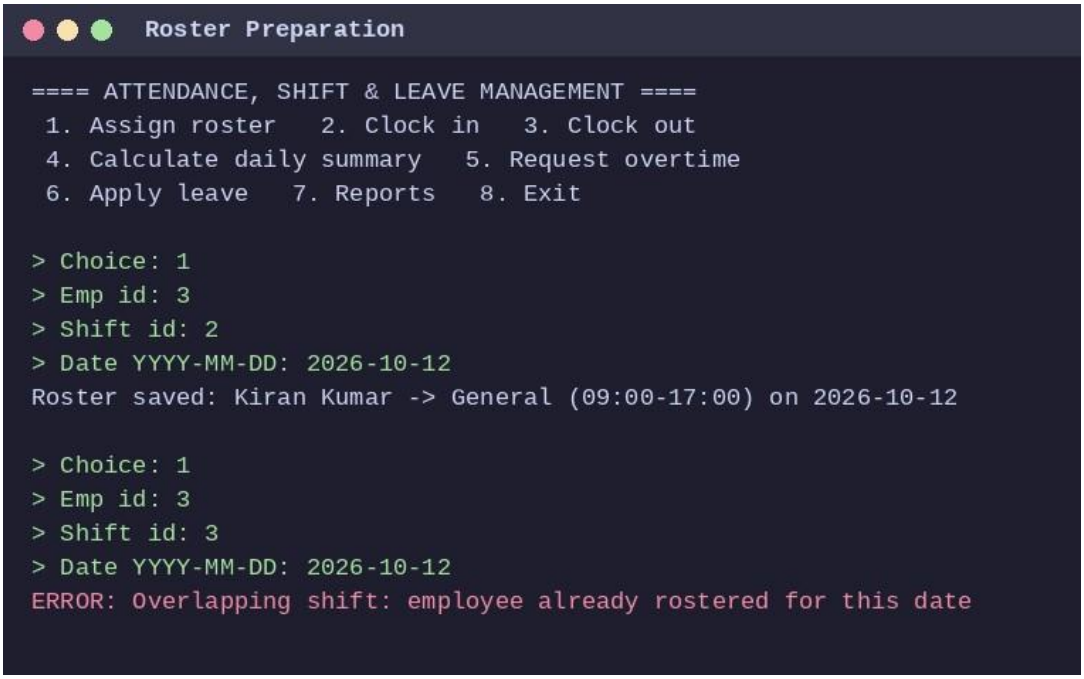


Figure 2. Roster Preparation and Overlapping-Shift Rejection

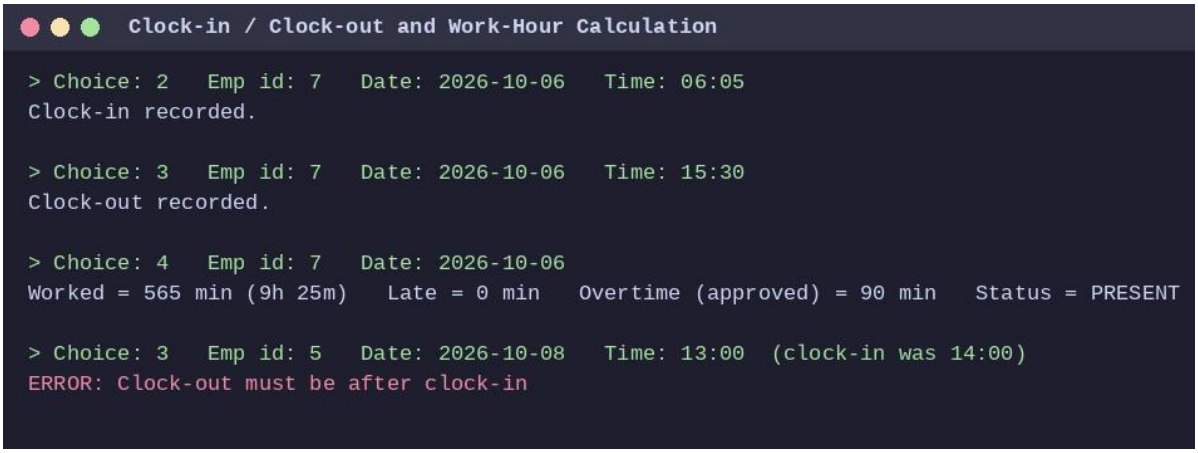


Figure 3. Clock-in, Clock-out and Work-Hour Calculation

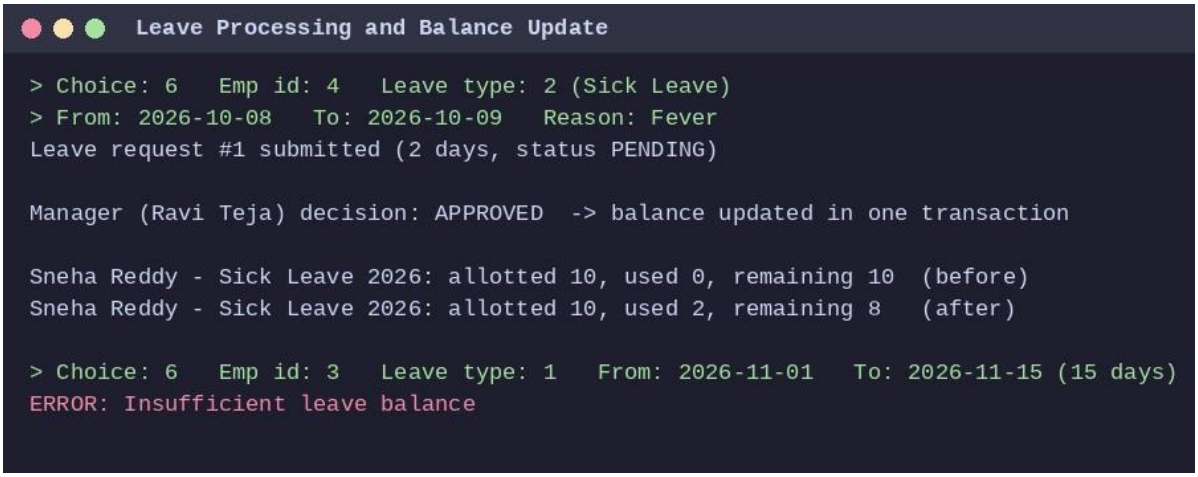


Figure 4. Leave Processing and Balance Update



Figure 5. Attendance Reports: Lateness and Department Summary

3.8 Testing and Results

Table 15 lists fourteen test cases, both positive and negative. The actual results were captured by running run_demo.py against a freshly created database.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Roster assignment	Assign Emp 3 to General shift on 2026-10-12	Accepted	Accepted	Pass
TC-02	No overlapping shifts	Assign Emp 3 a second shift on 2026-10-12	Rejected: Overlapping shift	Rejected: Overlapping shift: employee already rostered for this date	Pass
TC-03	Clock-in / clockout	Emp 6 clock-in 09:00 and clock-out 17:00 on 202610-08	Accepted	Accepted	Pass
TC-04	Clock-out after clock-in	Emp 5 clock-in 14:00, clock-out 13:00 on 202610-08	Rejected: clock-out must be later	Rejected: Clock-out must be after clock-in	Pass
TC-05	Authorised overtime	Insert summary with 60 min OT for Emp 4 with no approved request	Rejected: Overtime not authorised	Rejected: Overtime not authorised: no approved overtime request	Pass
TC-06	One daily summary	Calculate summary again for Emp 3 on 2026-10-05	Rejected: duplicate summary	Rejected: UNIQUE constraint failed: attendance_summary.emp_id, attendance_summary.work_date	Pass
Test ID	**Feature**	**Input / Action**	**Expected Result**	**Actual Result**	**Status**
TC-07	Leave within balance	Emp 3 applies 11 days Casual Leave (balance 9 left)	Rejected: Insufficient leave balance	Rejected: Insufficient leave balance	Pass
TC-08	Leave approval and balance update	Manager approves Emp 4 Sick Leave for 2 days	Accepted; used = 2	Accepted; result: 2	Pass
TC-09	Foreign key integrity	Insert employee with department 99	Rejected: FOREIGN KEY constraint	Rejected: FOREIGN KEY constraint failed	Pass
TC-10	Unique email	Insert employee with existing e-mail	Rejected: UNIQUE constraint	Rejected: UNIQUE constraint failed: employee.email	Pass
TC-11	Transaction rollback	Approve the same leave request twice	Rejected: already processed	Rejected: second approval blocked; transaction rolled back	Pass

TC-12	Create and search employee	Add Farhan Ali (Customer Care) and search 'Farhan'	Accepted; 1 matching row	Accepted; result: 1	Pass
TC-13	Update employee	Change designation of Emp 7 to Senior Technician	Accepted; designation updated	Accepted; result: Senior Technician	Pass
TC-14	Controlled removal	Deactivate Emp 8 and remove roster entry of Emp 3 for 2026-10-12	Accepted; Emp 8 INACTIVE, roster row removed	Accepted; result: INACTIVE	Pass

All fourteen test cases passed. Valid operations were accepted and every invalid operation was rejected with a clear message, showing that the application uses the designed database and that the constraints and triggers enforce the key business rules: no overlapping shifts, clock-out after clock-in, leave within balance, authorised overtime and one daily summary per employee. The rollback test showed that a repeated approval leaves the balance unchanged.

CHAPTER 4 CONCLUSION

This project addressed the problem of unreliable, manually consolidated attendance, shift and leave records. A thirteen-entity ER model was converted into a relational schema in Third Normal Form and implemented in SQLite with keys, constraints, views and triggers. A Python application built on this schema supports roster preparation, clock-in and clock-out, work-hour calculation, overtime approval, leave processing with balance update, employee maintenance and eight reports.

Testing confirmed that the database itself rejects overlapping shifts, invalid punches, excess leave and unauthorised overtime, and that approvals are processed atomically. The main learning was that placing rules in the database keeps data consistent for every program that uses it, and that normalization together with views gives accurate, payroll-ready reports.

4.1 Limitations and Future Enhancements

- The prototype uses SQLite and a console interface; moving to MySQL or PostgreSQL would require rewriting the trigger syntax, and a web or desktop front end would improve usability.
- Authentication is limited to stored user roles; password hashing, login screens and rolebased access checks in the application should be added.
- Biometric device integration, automatic daily summary generation, weekend and holiday handling in the status calculation, and payroll export are not yet implemented.
- Roster generation is manual; automatic roster suggestions based on coverage requirements could be added.

REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377-387, Jun. 1970.

[2] P. P. Chen, "The entity-relationship model—toward a unified view of data," ACM Trans. Database Syst., vol. 1, no. 1, pp. 9-36, Mar. 1976.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.

[6] SQLite Consortium, "SQLite documentation," SQLite. Accessed: Oct. 8, 2026. [Online]. Available: https://www.sqlite.org/docs.html

[7] Python Software Foundation, "sqlite3 — DB-API 2.0 interface for SQLite databases," Python 3 Documentation. Accessed: Oct. 8, 2026. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

APPENDICES

Appendix A: Complete Database Script (schema.sql)

```
PRAGMA foreign_keys = ON;

CREATE TABLE department (
    dept_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    dept_name   TEXT NOT NULL UNIQUE,
    location    TEXT NOT NULL DEFAULT 'Main Campus'
);

CREATE TABLE employee (
    emp_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    emp_name    TEXT NOT NULL,
    email       TEXT NOT NULL UNIQUE CHECK (email LIKE '%_@_%._%'),
    phone       TEXT,
designation TEXT NOT NULL,
dept_id     INTEGER NOT NULL,
    manager_id           INTEGER,
join_date   TEXT NOT NULL,
    status      TEXT NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
        ('ACTIVE','INACTIVE')),
    FOREIGN KEY (dept_id) REFERENCES department(dept_id) ON UPDATE CASCADE ON
        DELETE RESTRICT,
    FOREIGN KEY (manager_id) REFERENCES employee(emp_id) ON DELETE SET NULL
);

CREATE TABLE app_user (
    user_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    emp_id      INTEGER NOT NULL UNIQUE,
username    TEXT NOT NULL UNIQUE,
password_hash TEXT NOT NULL,
    role        TEXT NOT NULL CHECK (role IN ('ADMIN','MANAGER','EMPLOYEE')),
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE
);

CREATE TABLE shift (
    shift_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    shift_name  TEXT NOT NULL UNIQUE,
start_time  TEXT NOT NULL,
end_time    TEXT NOT NULL,
    grace_min   INTEGER NOT NULL DEFAULT 10 CHECK (grace_min >= 0)
);

CREATE TABLE roster (
    roster_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    emp_id      INTEGER NOT NULL,
shift_id    INTEGER NOT NULL,
work_date   TEXT NOT NULL,
    FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE,
FOREIGN KEY (shift_id) REFERENCES shift(shift_id) ON DELETE RESTRICT
);
```

```
-- Rule: no overlapping shifts. An employee may hold only one rostered shift per
-- day, and a shift on consecutive days may not overlap the previous night
shift.
CREATE UNIQUE INDEX uq_roster_emp_date ON roster(emp_id, work_date);
```

```
CREATE TRIGGER trg_roster_no_overlap
```

```
BEFORE INSERT ON roster
```

```
BEGIN
```

```
SELECT RAISE(ABORT, 'Overlapping shift: employee already rostered for this
```

```
        date')
```

```
WHERE EXISTS (SELECT 1 FROM roster r
```

```
        WHERE r.emp_id = NEW.emp_id AND r.work_date = NEW.work_date);
```

```
-- previous-day night shift (end_time earlier than start_time) spills into
```

```
today
```

```
SELECT RAISE(ABORT, 'Overlapping shift: previous night shift runs into this
```

```
        shift')
```

```
WHERE EXISTS (
```

```
SELECT 1
```

```
        FROM roster r
```

```
        JOIN shift ps ON ps.shift_id = r.shift_id
```

```
    JOIN shift ns ON ns.shift_id = NEW.shift_id
```

```
        WHERE r.emp_id = NEW.emp_id
```

```
        AND r.work_date = date(NEW.work_date, '-1 day')
```

```
        AND ps.end_time < ps.start_time
```

```
        AND ps.end_time > ns.start_time);
```

```
END;
```

```
CREATE TABLE attendance_punch (
```

```
punch_id    INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
   emp_id      INTEGER NOT NULL,
```

```
work_date   TEXT NOT NULL,
```

```
clock_in    TEXT NOT NULL,
```

```
  clock_out   TEXT,
```

```
    source      TEXT NOT NULL DEFAULT 'APP' CHECK (source IN
```

```
        ('APP','BIOMETRIC','MANUAL')),
```

```
FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE,
```

```
CHECK (clock_out IS NULL OR clock_out > clock_in)
```

```
);
```

```
CREATE INDEX idx_punch_emp_date ON attendance_punch(emp_id, work_date);
```

```
-- Rule: clock-out must be after clock-in (also guarded on UPDATE)
```

```
CREATE TRIGGER trg_punch_out_after_in
```

```
BEFORE UPDATE OF clock_out ON attendance_punch
```

```
WHEN NEW.clock_out IS NOT NULL AND NEW.clock_out <= OLD.clock_in
```

```
BEGIN
```

```
   SELECT RAISE(ABORT, 'Clock-out must be after clock-in');
```

```
END;
```

```
CREATE TABLE attendance_summary (
```

```
summary_id  INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
   emp_id      INTEGER NOT NULL,
```

```
work_date   TEXT NOT NULL,
```

```
  worked_min  INTEGER NOT NULL DEFAULT 0 CHECK (worked_min >= 0),
```

```
late_min    INTEGER NOT NULL DEFAULT 0 CHECK (late_min >= 0),
```

```
overtime_min INTEGER NOT NULL DEFAULT 0 CHECK (overtime_min >= 0),
```

```
   day_status  TEXT NOT NULL CHECK (day_status IN
```

```
        ('PRESENT','LATE','ABSENT','ON_LEAVE','HOLIDAY','WEEKOFF')),
```

```
-- Rule: one daily summary per employee
```

```
    UNIQUE (emp_id, work_date),
```

```
FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE overtime_request (
```

```
ot_id       INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
   emp_id      INTEGER NOT NULL,
```

```
work_date   TEXT NOT NULL,
```

```
requested_min INTEGER NOT NULL CHECK (requested_min > 0 AND requested_min <=
```

```
        240),
```

```
  reason      TEXT NOT NULL,
```

```
  status      TEXT NOT NULL DEFAULT 'PENDING' CHECK (status IN
```

```
        ('PENDING','APPROVED','REJECTED')),
```

```
    approved_by INTEGER,
```

```
FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE,
```

```
FOREIGN KEY (approved_by) REFERENCES employee(emp_id) ON DELETE SET NULL
```

```
);
```

```
CREATE TABLE leave_type (
```

```
   leave_type_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    type_name   TEXT NOT NULL UNIQUE,
```

```
annual_quota INTEGER NOT NULL CHECK (annual_quota >= 0),
```

```
is_paid     INTEGER NOT NULL DEFAULT 1 CHECK (is_paid IN (0,1))
```

```
);
```

```
CREATE TABLE leave_balance (
```

```
balance_id  INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
   emp_id      INTEGER NOT NULL,
```

```
leave_type_id INTEGER NOT NULL,
```

```
year        INTEGER NOT NULL,
```

```
   allotted    INTEGER NOT NULL CHECK (allotted >= 0),
```

```
used        INTEGER NOT NULL DEFAULT 0 CHECK (used >= 0),
```

```
    UNIQUE (emp_id, leave_type_id, year),
```

```
CHECK (used <= allotted),
```

```
FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE,
```

```
FOREIGN KEY (leave_type_id) REFERENCES leave_type(leave_type_id) ON DELETE
```

```
    RESTRICT
```

```
);
```

```
CREATE TABLE leave_application (
```

```
leave_id    INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
   emp_id      INTEGER NOT NULL,
```

```
leave_type_id INTEGER NOT NULL,
```

```
from_date   TEXT NOT NULL,
```

```
to_date     TEXT NOT NULL,
```

```
    days        INTEGER NOT NULL CHECK (days > 0),
```

```
  reason      TEXT,
```

```
  status      TEXT NOT NULL DEFAULT 'PENDING' CHECK (status IN
```

```
        ('PENDING','APPROVED','REJECTED')),
```

```
CHECK (to_date >= from_date),
```

```
FOREIGN KEY (emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE,
```

```
FOREIGN KEY (leave_type_id) REFERENCES leave_type(leave_type_id) ON DELETE
```

```
    RESTRICT
```

```
);
```

```
-- Rule: leave within balance (checked at application time)
```

```
CREATE TRIGGER trg_leave_within_balance
```

```
BEFORE INSERT ON leave_application
```

```
BEGIN
```

```
   SELECT RAISE(ABORT, 'Insufficient leave balance')
```

```
WHERE COALESCE((SELECT allotted - used FROM leave_balance
```

```
                    WHERE emp_id = NEW.emp_id
```

```
                      AND leave_type_id = NEW.leave_type_id
                      AND year = CAST(substr(NEW.from_date,1,4) AS INTEGER)), 0)
                          < NEW.days;
END;

CREATE TABLE approval (
    approval_id INTEGER PRIMARY KEY AUTOINCREMENT,
    request_type TEXT NOT NULL CHECK (request_type IN ('LEAVE','OVERTIME')),
    request_id  INTEGER NOT NULL,
approver_id INTEGER NOT NULL,
    decision    TEXT NOT NULL CHECK (decision IN ('APPROVED','REJECTED')),
    remarks     TEXT,
    decided_on  TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (approver_id) REFERENCES employee(emp_id) ON DELETE RESTRICT
);

-- Rule: overtime may be recorded only if it was authorised (APPROVED request
    exists)
CREATE TRIGGER trg_overtime_authorised
BEFORE INSERT ON attendance_summary
WHEN NEW.overtime_min > 0
BEGIN
    SELECT RAISE(ABORT, 'Overtime not authorised: no approved overtime request')
    WHERE NOT EXISTS (SELECT 1 FROM overtime_request o
                      WHERE o.emp_id = NEW.emp_id AND o.work_date = NEW.work_date
                        AND o.status = 'APPROVED' AND o.requested_min >=
                            NEW.overtime_min);
END;

CREATE TABLE holiday (
    holiday_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    holiday_date TEXT NOT NULL UNIQUE,
holiday_name TEXT NOT NULL,
    is_optional INTEGER NOT NULL DEFAULT 0 CHECK (is_optional IN (0,1))
);

CREATE VIEW v_daily_attendance AS
SELECT s.work_date, e.emp_id, e.emp_name, d.dept_name, s.day_status,
       s.worked_min, s.late_min, s.overtime_min
FROM attendance_summary s
JOIN employee e ON e.emp_id = s.emp_id
JOIN department d ON d.dept_id = e.dept_id;

CREATE VIEW v_leave_balance AS
SELECT e.emp_id, e.emp_name, lt.type_name, b.year, b.allotted, b.used,
       (b.allotted - b.used) AS remaining
FROM leave_balance b
JOIN employee e ON e.emp_id = b.emp_id
JOIN leave_type lt ON lt.leave_type_id = b.leave_type_id;
```

Appendix B: Source Code Reference

The following files are submitted with this report. The program can be run with python3 run_demo.py (creates the database, loads the sample data, executes all test cases and reports) and python3 app.py (interactive menu).

File	Description
schema.sql	Complete DDL: tables, constraints, triggers and views (Appendix A)
app.py	Python application: connection, seed data, roster, punches, work-hour calculation, overtime, leave, employee maintenance, reports and menu
run_demo.py	Loads sample data, executes the fourteen test cases and the reports, and writes results.json
attendance.db	SQLite database file generated by the scripts

Appendix C: Supplementary Report Outputs

Weekly Roster (JOIN)

```
+------------+--------------+---------------+------------+-------------+
| work_date  | emp_name     | dept_name     | shift_name | timing      |
+------------+--------------+---------------+------------+-------------+
| 2026-10-05 | Kiran Kumar  | Operations    | Morning    | 06:00-14:00 |
| 2026-10-05 | Vikram Singh | Operations    | Morning    | 06:00-14:00 |
| 2026-10-05 | Sneha Reddy  | IT Support    | General    | 09:00-17:00 |
| 2026-10-05 | Imran Khan   | Customer Care | Evening    | 14:00-22:00 |
| 2026-10-05 | Pooja Sharma | Customer Care | Evening    | 14:00-22:00 |
| 2026-10-06 | Kiran Kumar  | Operations    | Morning    | 06:00-14:00 |
| 2026-10-06 | Vikram Singh | Operations    | Morning    | 06:00-14:00 |
| 2026-10-06 | Sneha Reddy  | IT Support    | General    | 09:00-17:00 |
| 2026-10-06 | Imran Khan   | Customer Care | Evening    | 14:00-22:00 |
| 2026-10-06 | Pooja Sharma | Customer Care | Evening    | 14:00-22:00 |
+------------+--------------+---------------+------------+-------------+
```

Absence Report (NOT EXISTS subquery)

```
+------------+--------------+
| work_date  | emp_name     |
+------------+--------------+
| 2026-10-06 | Pooja Sharma |
| 2026-10-08 | Kiran Kumar  |
| 2026-10-08 | Vikram Singh |
+------------+--------------+
```

Overtime Summary (GROUP BY + SUM)

```
+--------------+--------------+---------+
| emp_name     | total_ot_min | ot_days |
+--------------+--------------+---------+
| Vikram Singh | 90           | 1       |
+--------------+--------------+---------+
```

Leave Balance (VIEW)

```
+-------------+--------------+----------+------+-----------+
| emp_name    | type_name    | allotted | used | remaining |
+-------------+--------------+----------+------+-----------+
| Kiran Kumar | Casual Leave | 12       | 3    | 9         |
| Kiran Kumar | Sick Leave   | 10       | 0    | 10        |
| Sneha Reddy | Casual Leave | 12       | 0    | 12        |
| Sneha Reddy | Sick Leave   | 10       | 2    | 8         |
+-------------+--------------+----------+------+-----------+
```

Shift Coverage (GROUP BY + HAVING)

```
+------------+------------+-------------+
| work_date  | shift_name | staff_count |
+------------+------------+-------------+
| 2026-10-05 | Evening    | 2           |
| 2026-10-05 | General    | 1           |
| 2026-10-05 | Morning    | 2           |
+------------+------------+-------------+
```

Employees above average worked time (nested query)

```
+--------------+------------+------------+
| emp_name     | work_date  | worked_min |
+--------------+------------+------------+
| Vikram Singh | 2026-10-06 | 565        |
+--------------+------------+------------+
```